

QC and Read Mapping

Ioana Lemnian, PathoNext GmbH
ioana.lemnian@pathonext.de

Agenda

- Recap NGS Vocabulary / Data QC
- Read trimming
- Read mapping
 - NGS Alignment
 - Data Formats
 - Mapping statistics
 - Visualization

Recap

- Library preparation
- Index/barcode
- # fragments ~ # clusters ~ # read pairs ~ # paired-end reads
- Flow cell
- Demultiplexing
- Q30

Additional resources

- **Patterned vs unpatterned flow cell**

https://www.youtube.com/watch?v=QnMKfWbk_hY

- **Unique molecular identifiers (UMI)**

<https://www.lexogen.com/rna-lexicon-what-are-unique-molecular-identifiers-umis-and-why-do-we-need-them/>

<https://dnatech.genomecenter.ucdavis.edu/faqs/what-are-umis-and-why-are-they-used-in-high-throughput-sequencing/>

- ***-Seq**

<https://www.illumina.com/content/dam/illumina-marketing/documents/applications/ngs-library-prep/ForAllYouSeqMethods.pdf>

<https://liorpachter.wordpress.com/seq/>

<https://github.com/qliugithub/Bioinformatics-Resources>

Software

- MultiQC

```
conda install bioconda::multiqc
```

- Trimmomatic

```
conda install bioconda::trimmomatic
```

- bwa/ bwa-mem2

```
conda install bioconda::bwa-mem2
```

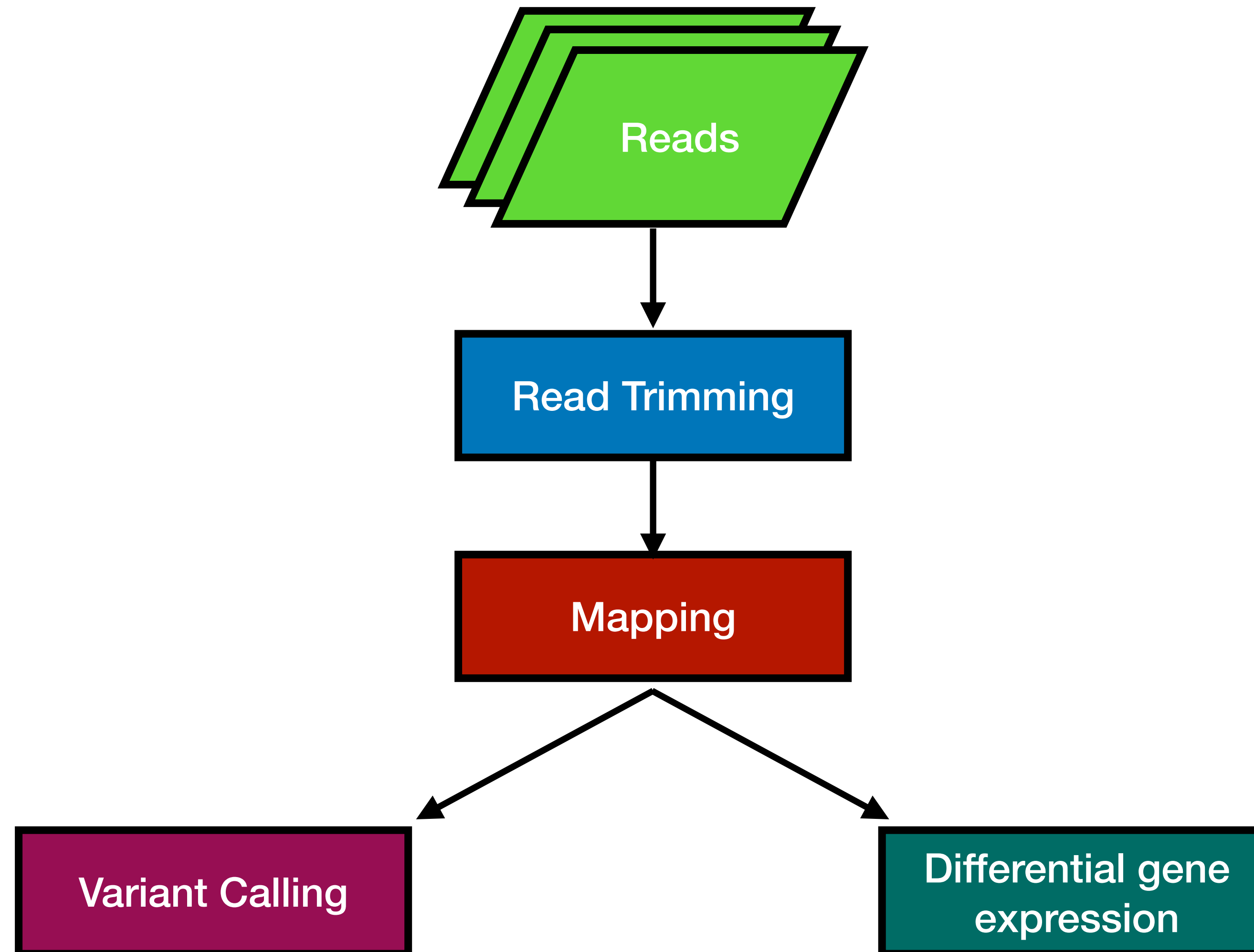
- samtools (<https://www.htslib.org>)

```
conda install bioconda::samtools
```

- Integrative Genomics Viewer (IGV)

```
conda install bioconda::igv
```

NGS Workflows



Read pre-processing

- Removal of adapter sequences
- Removal of low quality reads
- Removal of certain low quality bases

Read pre-processing

Trimmomatic

```
trimmomatic PE -phred33 input_R1.fq.gz input_R2.fq.gz output_R1_paired.fq.gz  
output_R1_unpaired.fq.gz output_R2_paired.fq.gz output_R2_unpaired.fq.gz  
ILLUMINACLIP:TruSeq3-PE.fa:2:30:10 LEADING:3 TRAILING:3 SLIDINGWINDOW:4:15  
MINLEN:36
```

- Remove adapters (ILLUMINACLIP:TruSeq3-PE.fa:2:30:10)
- Remove leading low quality or N bases (below quality 3) (LEADING:3)
- Remove trailing low quality or N bases (below quality 3) (TRAILING:3)
- Scan the read with a 4-base wide sliding window, cutting when the average quality per base drops below 15 (SLIDINGWINDOW:4:15)
- Drop reads below the 36 bases long (MINLEN:36)

Before and after trimming

- Run fastQC on the 6484_snippet_*.fastq
- Run multiQC on both reports
- Run trimmomatic on the samples
- Run fastQC again on the trimmed data

Read Mapping

Problem: Find the locus of origin of each read within a reference genome

- sequence alignment problem, with one sequence being short (150bp) and one sequence being extremely long (up to several GB)
- there are millions of reads to be mapped
- the read sequences contain errors and artefacts
- the reference genomes have a highly variable quality

Reference genomes

The reference genomes are far from being perfect:

- missing regions
- repeats / low complexity regions
- genome rearrangements
- the level of manual curation varies greatly
- technical artefacts from the assembly steps not removed
- contaminations might be present

Reference genomes

Sources: Genome data base from NCBI

Human Genome

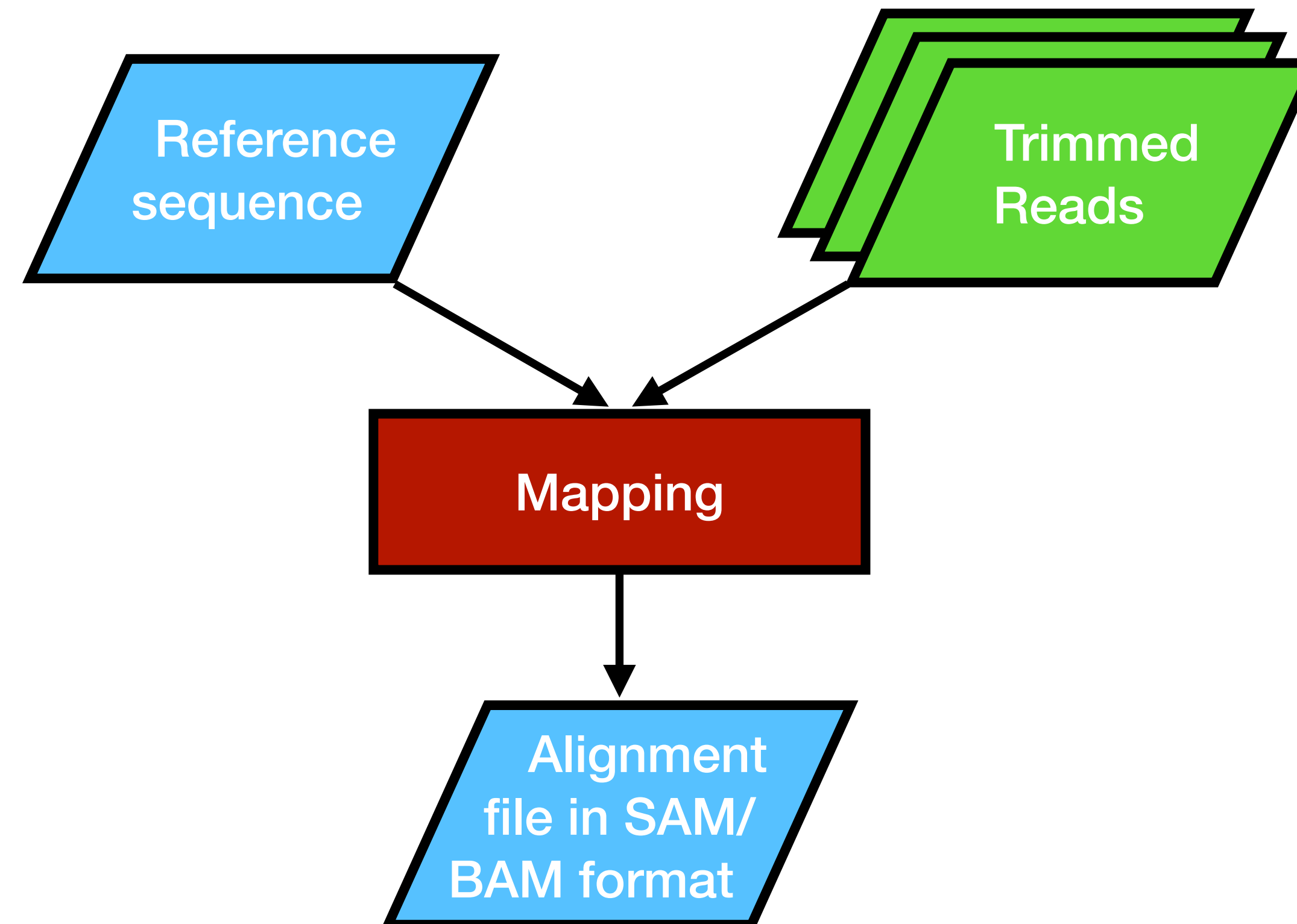
- multiple versions are being used (GRCh37/hg19 or GRCh38/hg38)
- Dec. 2022 full sequence of Y-chromosome
- May 2023 Human Pangenome Reference
- multiple annotations (Ensembl/GENCODE , RefSeq)

<https://www.ncbi.nlm.nih.gov/genome/guide/human/>

<https://www.genome.gov/human-genome-project/timeline>

Read Mapping

Problem: Find the locus of origin of each read within a reference genome



Read Mapping

Problem: Find the locus of origin of each read within a reference genome

STEP 1: create an index of the complete genome

STEP 2: fast exact search for read substrings / seeds / k-mers

STEP 3: elongate the matched seeds

STEP 4: perform local alignments

STEP 5: remove alignments with more mismatches than allowed

STEP 6: compare all alignments and return the ones with minimum number of errors.

Data Formats

- Reference: .fasta
- Reads: .fastq, .fastq.gz
- Alignment:
 - SAM - Sequence Alignment Map
 - BAM - Binary Alignment Map
 - CRAM - Compressed Alignment Map
- <https://samtools.github.io/hts-specs/>

SAM Format

- Header section starting with “@”
- Alignment section: Mandatory fields

Col	Field	Type	Regexp/Range	Brief description
1	QNAME	String	[!-?A-~]{1,254}	Query template NAME
2	FLAG	Int	$[0, 2^{16} - 1]$	bitwise FLAG
3	RNAME	String	* [:rname:^*=] [:rname:]*	Reference sequence NAME ¹¹
4	POS	Int	$[0, 2^{31} - 1]$	1-based leftmost mapping POSition
5	MAPQ	Int	$[0, 2^8 - 1]$	MAPping Quality
6	CIGAR	String	* ([0-9]+[MIDNSHPX=])+	CIGAR string
7	RNEXT	String	* = [:rname:^*=] [:rname:]*	Reference name of the mate/next read
8	PNEXT	Int	$[0, 2^{31} - 1]$	Position of the mate/next read
9	TLEN	Int	$[-2^{31} + 1, 2^{31} - 1]$	observed Template LENgth
10	SEQ	String	* [A-Za-z=.]+	segment SEQUENCE
11	QUAL	String	[!-~]+	ASCII of Phred-scaled base QUALity+33

¹¹Reference sequence names may contain any printable ASCII characters with the exception of certain punctuation characters, and may not start with ‘*’ or ‘=’. See Section 1.2.1 for details and an explanation of the [:rname:] notation.

Source: [Sequence Alignment/Map Format Specification](#)

SAM Format - Flag

- Bitwise flag, each bit related to a specific feature

Bit		Description
1	0x1	template having multiple segments in sequencing
2	0x2	each segment properly aligned according to the aligner
4	0x4	segment unmapped
8	0x8	next segment in the template unmapped
16	0x10	SEQ being reverse complemented
32	0x20	SEQ of the next segment in the template being reverse complemented
64	0x40	the first segment in the template
128	0x80	the last segment in the template
256	0x100	secondary alignment
512	0x200	not passing filters, such as platform/vendor quality controls
1024	0x400	PCR or optical duplicate
2048	0x800	supplementary alignment

- Decoding SAM flags: <https://broadinstitute.github.io/picard/explain-flags.html>

Source: [Sequence Alignment/Map Format Specification](#)

SAM Format - CIGAR

- Describes the alignment

Op	BAM	Description	Consumes query	Consumes reference
M	0	alignment match (can be a sequence match or mismatch)	yes	yes
I	1	insertion to the reference	yes	no
D	2	deletion from the reference	no	yes
N	3	skipped region from the reference	no	yes
S	4	soft clipping (clipped sequences present in SEQ)	yes	no
H	5	hard clipping (clipped sequences NOT present in SEQ)	no	no
P	6	padding (silent deletion from padded reference)	no	no
=	7	sequence match	yes	yes
X	8	sequence mismatch	yes	yes

Source: [Sequence Alignment/Map Format Specification](#)

Read Mapping

- **multiple tools are available for mapping**
- gapped alignment (mostly for DNA)
 - **bwa**
- spliced alignment (mostly for RNA)
 - **STAR**

Read Mapping with BWA

- Index the reference (genome) sequence

```
bwa-mem2 index [-p prefix] reference.fa
```

- Perform the alignment

```
bwa-mem2 mem -t <num_threads> <prefix> <reads.fq/fa> >  
out.sam
```

- Convert SAM to BAM for sorting

```
samtools view -S -b out.sam > out.bam
```

- Sort BAM for SNP calling

```
samtools sort out.bam out-sorted
```

- Index BAM (it will generate a file out-sorted.bam.bai)

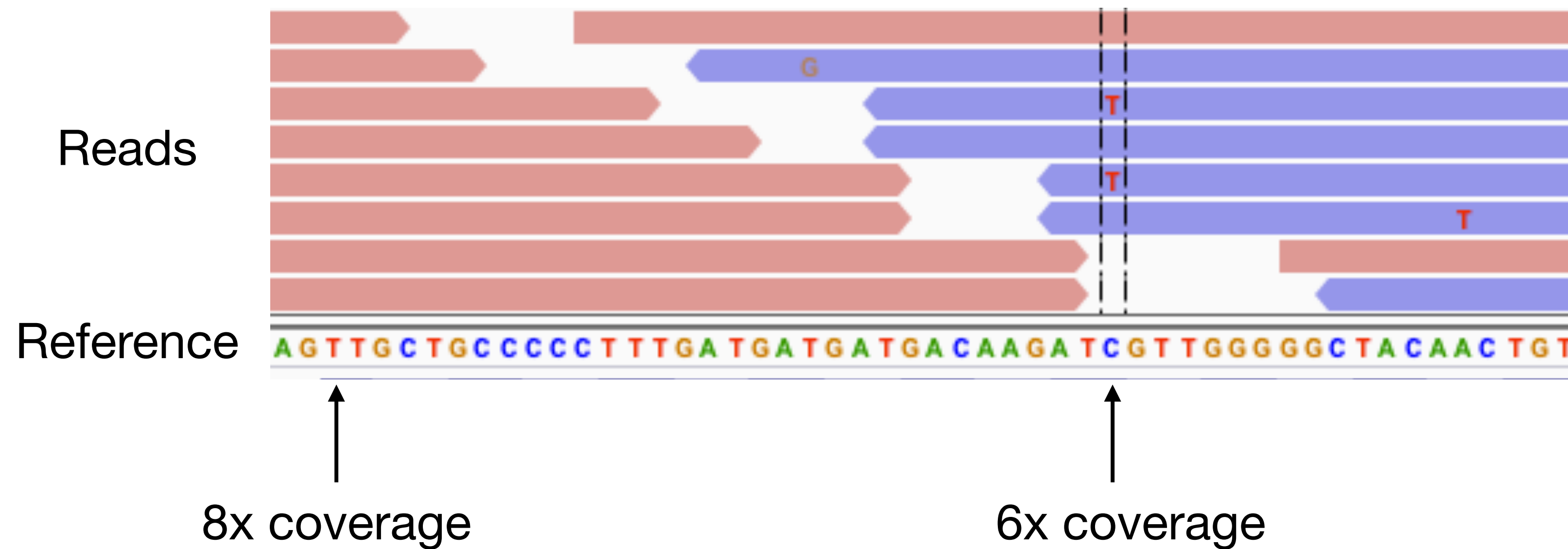
```
samtools index out-sorted.bam
```

Mapping statistics

- Mapping rate = total percentage of reads mapped to the reference
- Uniquely mapped reads = reads mapping to only one region in the reference
- Discordantly mapped reads = read-pairs that are mapping not within the expectation (R1 forward, R2 reverse, expected insert length)
- Mapped single reads = pairs of whom only one read is mapping to the reference, while the mate is not mapped.

Coverage

- Slightly different meaning depending on the target region.
- For **a position**, the coverage represents *the number of reads covering that position*:



Coverage

- For a **genome**, the coverage is given by *the total number of sequenced bases divided by the size of the genome*.

coverage = $(N \times L) / G$, where N = number of mapped reads
L = length of reads
G = genome size

- For a **locus**, the coverage represents *the number of reads covering that locus divided by the size of the locus*

Coverage

- For a **genome**, the coverage is given by *the total number of sequenced bases divided by the size of the genome*.

$$\text{coverage} = (N \times L) / G, \text{ where } N = \text{number of mapped reads}$$

L = length of reads
G = genome size

- For a **locus**, the coverage represents *the number of reads covering that locus divided by the size of the locus*

Important: The coverage represents an average value, but does not give any information about the uniformity of the read distribution!

Coverage - An example

- What coverage of the human genome is expected with 300M paired-end reads (2x150bp)?

$$\text{coverage} = 300M \times (2 \times 150 \text{ bp}) / 3 \text{ GB}$$

$$= 300 \times 10^6 \times 300 \text{ bp} / (3 \times 10^9 \text{ bp})$$

$$= 30$$

- Given the size of the banana genome is 523 MB, how many paired-end reads (2x150bp) are needed to achieve 50x coverage?